

★ Member-only story

Nicegui +

Python +

Internal Tools +

Architecture +

Productivity +

Why NiceGUI Is Perfect for Internal Business Apps

Stop Building React Dashboards Nobody Asked For



Yamishift

Following ▾

7 min read · Jul 18, 2026



39



2



Discover why NiceGUI has become one of the most productive Python frameworks for building internal business applications, dashboards, admin portals, and operational tools without maintaining a separate frontend.

Everyone Wants a Beautiful Frontend...

...until they're the one maintaining it.

A few years ago, our internal tooling stack looked “modern.”

- FastAPI backend
- React frontend
- TypeScript
- Redux
- Material UI
- Docker
- CI/CD
- Authentication
- OpenAPI
- Storybook

On paper, it looked impressive.

In reality?

The business needed a dashboard that displayed warehouse inventory.

We spent two weeks debating frontend state management.

That experience changed how I think about internal software.

Because internal applications have very different requirements than customer-facing products.

Employees don't care whether your table component uses virtual DOM optimizations.

They care that clicking **"Approve Invoice"** doesn't take six seconds.

They care that reports load.

They care that the search works.

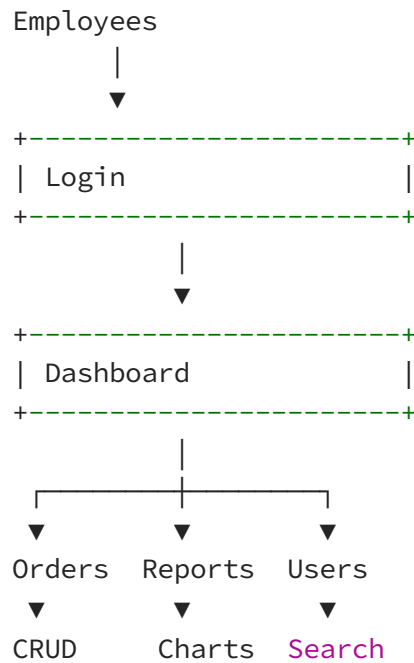
Everything else is secondary.

That's exactly where **NiceGUI** shines.

Instead of treating Python as "just the backend," NiceGUI lets Python own the entire application while the browser becomes a rendering engine. Under the hood it builds on FastAPI, Starlette, Uvicorn, Vue, Quasar, and Socket.IO, so you still get an asynchronous web stack without writing JavaScript yourself.

Internal Software Is a Different Engineering Problem

Most business applications look surprisingly similar.



Nobody is trying to build the next Instagram.

They're trying to answer questions like:

- Which invoices failed today?
- Which customers haven't paid?
- Which warehouse is running low?
- Which jobs are still processing?
- Which employees need approval?

Those are data problems.

Not animation problems.

One lesson I've learned repeatedly is this:

Internal applications fail because they slow down business processes — not because their buttons aren't animated.

Why NiceGUI Feels Different

Most Python web frameworks stop at the API.

```
graph TD; Browser --> React; React --> REST_API[REST API]; REST_API --> FastAPI; FastAPI --> Database;
```

NiceGUI removes an entire layer.

```
graph TD; Browser --> NiceGUI; NiceGUI --> FastAPI;
```

|
PostgreSQL

Your UI is simply Python.

Buttons.

Tables.

Dialogs.

Charts.

Everything stays inside one language.

That sounds like a small change.

It isn't.

It removes an enormous amount of cognitive overhead. NiceGUI follows a backend-first approach while allowing you to expose REST endpoints, mount additional ASGI apps, or integrate directly with existing FastAPI applications when needed.

A Real Internal Tool

Imagine operations staff need to monitor customer orders.

[Open in app](#)**Medium**

Search



Write



frontend/
backend/
shared-types/

we can start with

```
order_portal/  
|  
├─ app.py  
├─ database.py  
├─ models.py  
├─ repositories.py  
├─ services.py  
├─ auth.py  
├─ ui/  
│   ├─ dashboard.py  
│   ├─ orders.py  
│   └─ users.py  
└─ requirements.txt
```

One language.

One deployment.

One Docker image.

One debugging experience.

For a surprising number of companies...

that's enough.

Building Something Real

Let's skip the "Hello World."

Suppose we're building an operations dashboard.

First, configure the database.

```
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    create_async_engine,
    async_sessionmaker,
)

DATABASE_URL = (
    "postgresql+asyncpg://admin:secret@db/orders"
)

engine = create_async_engine(
    DATABASE_URL,
    pool_size=20,
    max_overflow=10,
    echo=False,
)

AsyncSessionLocal = async_sessionmaker(
    engine,
    expire_on_commit=False,
)

async def get_session() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session
```


Nothing unusual.

Because NiceGUI doesn't force a database layer.

You can continue using SQLAlchemy exactly as you already do.

That's a recurring theme with the framework:

| It replaces the frontend — not your backend architecture.

Domain Models Stay Clean

```
from sqlalchemy.orm import Mapped
from sqlalchemy.orm import mapped_column
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)

    customer: Mapped[str]

    amount: Mapped[float]

    status: Mapped[str]

    created_at: Mapped[datetime]
```

Notice what's missing.

No UI code.

No framework coupling.

No decorators from twenty different libraries.

Business models remain business models.

Repository Pattern

```
from sqlalchemy import select

class OrderRepository:

    def __init__(self, session: AsyncSession):
        self.session = session

    async def latest(self, limit: int = 50):

        stmt = (
            select(Order)
            .order_by(Order.created_at.desc())
            .limit(limit)
        )

        result = await self.session.execute(stmt)

        return result.scalars().all()
```

This layer doesn't know anything about NiceGUI.

That's intentional.

Frameworks come and go.

Domain logic shouldn't.

Service Layer

Business rules belong here.

```
class DashboardService:

    def __init__(self, repository: OrderRepository):
        self.repository = repository

    async def dashboard(self):

        orders = await self.repository.latest()

        revenue = sum(
            order.amount
            for order in orders
            if order.status == "PAID"
        )

        pending = sum(
            1
            for order in orders
            if order.status == "PENDING"
        )

        return {
            "orders": orders,
            "revenue": revenue,
            "pending": pending,
        }
```

At this point we've written a completely ordinary backend.

That's exactly what you want.

NiceGUI shouldn't force you to rethink your architecture.

It should simply become another presentation layer.

Now the Interesting Part

This is where most frameworks tell you to switch languages.

NiceGUI doesn't.

```
from nicegui import ui

service = DashboardService(repository)
@ui.page("/")
async def dashboard():

    data = await service.dashboard()

    ui.label("Operations Dashboard") \
        .classes("text-h4")

    with ui.row():

        ui.card().tight()

        with ui.card():

            ui.label("Revenue")

            ui.label(
```

```
f"${data['revenue']:.2f}"  
).classes("text-h5 text-green")  
  
with ui.card():  
  
    ui.label("Pending Orders")  
  
    ui.label(  
        str(data["pending"])  
    ).classes("text-h5")
```

No REST endpoint.

No Axios.

No JSON serialization.

No React hooks.

No frontend build process.

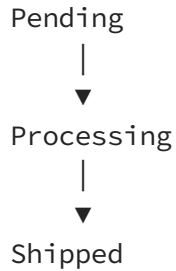
The UI talks directly to your Python objects, while NiceGUI handles the browser communication over its real-time architecture.

Real-Time Updates Without Writing JavaScript

One of the biggest surprises when you first use NiceGUI is that the UI doesn't feel like a traditional server-rendered application.

It feels live.

When an order changes state...



...the dashboard can update immediately without manually refreshing the page.

That's because NiceGUI maintains a real-time connection between the browser and your Python application using Socket.IO. Rather than sending every tiny UI change immediately, it batches updates efficiently and even supports reconnecting clients without requiring a full page reload.

Suppose an administrator approves an order.

```
from nicegui import ui

async def approve(order_id: int):

    await order_service.approve(order_id)

    ui.notify(
        f"Order #{order_id} approved",
        color="positive",
    )

    orders_table.refresh()
```

No REST endpoint.

No frontend polling.

No manual state synchronization.

The Python function updates the database.

NiceGUI updates the browser.

Sometimes the shortest code path is also the easiest to maintain.

Making Tables Actually Useful

Internal software usually revolves around tables.

Lots of tables.

Thousands of rows.

Hundreds of filters.

Instead of building dozens of React components, we can stay entirely inside Python.

```
from nicegui import ui

@ui.refreshable
```

```
async def orders_table():

    rows = await repository.latest(100)    ui.table(
        columns=[
            {
                "name": "id",
                "label": "Order",

                "field": "id",
            },
            {
                "name": "customer",
                "label": "Customer",
                "field": "customer",
            },
            {
                "name": "status",
                "label": "Status",
                "field": "status",
            },
            {
                "name": "amount",
                "label": "Amount",
                "field": "amount",
            },
        ],
        rows=[
            {
                "id": o.id,
                "customer": o.customer,
                "status": o.status,
                "amount": o.amount,
            }
            for o in rows
        ],
        pagination=25,
    )
```

The code remains surprisingly readable.

Even months later.

Don't Let the UI Touch the Database

One mistake I've seen teams make is writing SQL directly inside page functions.

Bad example:

```
@ui.page("/")
async def dashboard():

    async with AsyncSessionLocal() as session:

        result = await session.execute(
            text(
                "SELECT * FROM orders"
            )
        )

        ui.table(
            rows=result.mappings().all()
        )
```

This works.

Until it doesn't.

The UI now owns database access, business rules, and rendering.

Instead, keep the boundaries clear.

UI
|



The page should ask for data.

Not decide how data is fetched.

That separation makes testing dramatically easier.

Background Jobs Still Matter

Internal applications rarely end with CRUD.

Eventually someone asks for

- PDF exports
- Excel reports
- nightly imports
- invoice generation
- email notifications

Those shouldn't block the UI.

Celery works perfectly here.

```
from celery import shared_task

@shared_task(
    autoretry_for=(Exception,),
    retry_backoff=True,
    max_retries=5,
)
def generate_monthly_report(month: str):

    report_service.create(month)
```

The UI becomes refreshingly simple.

```
def export():

    generate_monthly_report.delay("2026-07")

    ui.notify(
        "Report generation started."
    )
```

Users don't care whether the task runs inside Celery, Dramatiq, or a background worker.

They care that clicking the button returns immediately.

Fast systems often feel faster because they know when **not** to wait.

Authentication Doesn't Need Reinvention

Every internal application eventually asks the same question.

“Who is allowed to see this?”

Since NiceGUI is built on FastAPI, you can reuse the same authentication and dependency injection patterns you already trust. It can also expose REST APIs alongside your UI when needed.

```
from fastapi import Depends

async def current_user():

    ...

@ui.page("/admin")
async def admin_page(

    user=Depends(current_user),

):

    if not user.is_admin:

        ui.label("Access denied")

        return

    ui.label(
        f"Welcome {user.name}"
    )
```

No duplicate authentication system.

No second identity provider.

One backend.

One source of truth.

Caching Expensive Queries

Business dashboards often execute the same reports repeatedly.

Instead of hammering PostgreSQL every refresh...

```
class DashboardService:

    async def metrics(self):

        cached = await redis.get(
            "dashboard_metrics"
        )

        if cached:

            return json.loads(cached)

        metrics = await self.calculate()

        await redis.setex(
            "dashboard_metrics",
            60,
            json.dumps(metrics),
        )

        return metrics
```

Even a 60-second cache can reduce database load dramatically.

Internal users usually don't need millisecond-perfect analytics.

They need dashboards that respond instantly.

Structured Logging

Production debugging becomes painful without consistent logs.

```
import structlog

logger = structlog.get_logger()

logger.info(
    "order_approved",
    order_id=order.id,
    customer=order.customer,
    user=current_user.email,
)
```

Later, when finance asks,

“Who approved this order three weeks ago?”

...the answer is already sitting in your logs.

Good observability saves far more engineering hours than premature optimization.

Health Checks

Every production deployment deserves health endpoints.

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/health")

async def health():

    return {
        "status": "ok",
        "database": "connected",
        "redis": "connected",
    }
```

Simple.

Predictable.

Container-friendly.

Whether you're deploying with Docker Compose or Kubernetes, those endpoints make orchestration much easier.

Docker Deployment

One of my favorite parts about NiceGUI projects is how boring deployment becomes.

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

One container.

One application.

One deployment pipeline.

No separate Node.js build stage.

No frontend artifact synchronization.

No reverse proxy gymnastics just to serve static assets.

That's a meaningful productivity win, especially for small engineering teams.

Nicegui +

Python +

Internal Tools +

Architecture +

Productivity +



Written by Yamishift

687 followers · 88 following

Following ▾

minor character writing about code, ai & side projects. build logs, experiments, and late-night notes from learning in public.

